ESCUELA DE INGENIERÍA DE FUENLABRADA

GRADO EN INGENIERÍA EN
TECNOLOGÍAS DE LA TELECOMUNICACIÓN

TRABAJO FIN DE GRADO

SISTEMA DE REPORTE SEGURO FRENTE A ROOTKITS
DE ESPACIO DE USUARIO

Autor: Noel Rodríguez Pérez

Tutor: Gorka Guardiola Múzquiz

Curso académico 2025-2026

Este trabajo se distribuye bajo los términos de la licencia internacional CC BY-NC-ND International License (Creative Commons Licencia Atribución/Reconocimiento-NoComercial-SinDerivados 4.0 International). Usted es libre de *compartir*: copiar y redistribuir el material en cualquier medio o formato. La licenciante no puede revocar estas libertades en tanto usted siga los términos de la licencia:

- *Atribución.* Usted debe dar crédito de manera adecuada, brindar un enlace a la licencia, e indicar si se han realizado cambios. Puede hacerlo en cualquier forma razonable, pero no de forma tal que sugiera que usted o su uso tienen el apoyo de la licenciante.
- *No Comercial.* Usted no puede hacer uso del material con fines comerciales.
- *Sin Derivadas.* Si remezcla, transforma o crea a partir del material, no podrá distribuir el material modificado.

  No hay restricciones adicionales — No puede aplicar términos legales ni medidas tecnológicas que restrinjan legalmente a otras a hacer cualquier uso permitido por la licencia.

No tiene que cumplir con la licencia para elementos del materiale en el dominio público o cuando su uso esté permitido por una excepción o limitación aplicable. No se dan garantías. La licencia podría no darle todos los permisos que necesita para el uso que tenga previsto. Por ejemplo, otros derechos como publicidad, privacidad, o derechos morales pueden limitar la forma en que utilice el material.

Documento de **Noel Rodríguez Pérez**.

Agradecimientos

En primer lugar, me gustaría agradecer a mi tutor, **Gorka Guardiola Múzquiz**, por su constante apoyo, paciencia y dedicación durante el desarrollo de este proyecto. Todo el conocimiento que me ha transmitido ha sido fundamental no solo para este trabajo, sino para saber el interés real que tengo por este campo de la Ciberseguridad. Gracias por enseñarme tanto.

A todos los docentes de mi carrera, por dotarme de conocimientos que me permiten ahora cerrar este ciclo y empezar mi carrera profesional de la mejor manera posible.

También, a todas las personas de mi entorno, por haberme acompañado en todo este proceso y haber disfrutado de su compañía y apoyo todo este tiempo.

Quiero agradecer especialmente a mi pareja, por su apoyo incondicional en mis decisiones y la gran ayuda que me ha aportado en mi desarrollo como persona. A mis padres y a mi hermana, por darme la educación y apoyo que me han hecho llegar a donde estoy actualmente. Y a mi grupo de la universidad, por haber hecho este ciclo mucho más ameno y haberme hecho disfrutar del proceso, incluso en momentos difíciles.

Muchas gracias por todo.

A mi familia, pareja y amigos;
mi mayor apoyo en toda esta etapa.

Madrid, 14 de febrero de 2026
Noel Rodríguez Pérez

Resumen

La seguridad en los sistemas modernos es un área en constante evolución, donde la integridad de los datos y el control de su funcionamiento juegan un papel fundamental. Esto es debido a la presencia de amenazas y agentes externos que trabajan con el fin malicioso de comprometer dicha información, haciendo necesario el desarrollo de mecanismos de protección cada vez más robustos.

En este ámbito, una de las amenazas más críticas para los sistemas actualmente son los *rootkits*, software malicioso diseñado para ocultarse en el núcleo del sistema (*kernel*) y controlar su funcionamiento. Al infiltrarse en los niveles más profundos, este *malware* adquiere permisos de administrador, permitiéndole controlar la ejecución de procesos de forma arbitraria y manipular respuestas e información del sistema operativo, impidiendo obtener una visión verídica del estado real de los archivos y procesos del sistema. Esto provoca una falta de integridad en la monitorización y control del mismo, donde por defecto se deposita una confianza implícita en una fuente de datos sensibles que puede estar comprometida ante dicho tipo de amenaza.

Para abordar esta problemática, en este proyecto se ha diseñado e implementado un *Módulo del Kernel* (*LKM*, *Loadable Kernel Module*) en un sistema *Linux* con el objetivo de protegerlo ante este tipo de amenazas. Esta solución introduce un mecanismo de autenticación criptográfica basado en HMAC (*Hash-based Message Authentication Code*) del contenido proporcionado por el *kernel*, mediante una clave simétrica secreta que solo es conocida por el administrador, garantizando la integridad de la información ofrecida.

Los experimentos realizados aproximan un entorno real de operación donde la información sensible es solicitada desde el *espacio de usuario* hacia el *espacio del kernel*. Con este fin, se define el contenido sensible del sistema a autenticar mediante un listado de todos los procesos en ejecución y sus respectivos metadatos, generado en el propio *Módulo del Kernel*, sirviendo como ejemplo de información crítica que puede ser

manipulada por un *rootkit*. Sobre este contenido, se calcula el HMAC con la respectiva clave secreta y se adjunta a la información proporcionada al usuario, de forma que este pueda verificar su integridad y autenticidad.

Para validar esta propuesta, se han desarrollado componentes de usuario que prueban la funcionalidad. Un primer componente genera un fichero cuyo descriptor es transferido al *kernel* para que este lo referencie, estableciendo así un canal de comunicación seguro. Por otro lado, la integridad de los datos reportados se confirma mediante un segundo componente de verificación que analiza la información escrita por el *kernel*, calculando localmente el HMAC del contenido con la misma clave secreta y contrastándolo posteriormente con el recibido para asegurar que coinciden y que el contenido es verídico.

Acrónimos

API *Application Programming Interface*

CIA *Confidentiality, Integrity and Availability*

CLI *Command Line Interface*

CPU *Central Processing Unit*

RAM *Random Access Memory*

FD *File Descriptor*

GNU *GNU's Not Unix*

OS *Operating System*

MAC *Message Authentication Code*

HMAC *Hash-based Message Authentication Code*

SHA *Symetric Hash Algorithm*

ID *Identifier*

LKM *Loadable Kernel Module*

PCB *Process Control Block*

US *User Space*

KS *Kernel Space*

NS *Namespace*

UID *User Identifier*

PID *Process Identifier*

GID *Group Identifier*

PS *Process Status*

VM *Virtual Memory*

VFS *Virtual File System*

FS *File System*

FUSE *Filesystem in Userspace*

FHS *Filesystem Hierarchy Standard*

MMU *Memory Management Unit*

BSS *Block Started by Symbol*

PC *Program Counter*

SP *Stack Pointer*

SYS *System*

BUF *Buffer*

GFP *Get Free Pages*

UTS *UNIX Time-Sharing*

IPC *Inter-Process Communication*

EDR *Endpoint Detection and Response*

HIDS *Host-based Intrusion Detection System*

PoLP *Principle of Least Privilege*

RAT *Remote Administration Tool*

Índice general

1. Introducción **1**
1.1. Ciberseguridad 2
1.1.1. Vulnerabilidades y Ataques 3
1.1.2. Defensas 5
1.1.3. Rootkits 6

2. Estado del Arte **8**
2.1. Sistemas operativos *Linux* 8
2.1.1. Arquitectura y anillos de privilegios 10
2.1.2. Memoria: Espacio de usuario y espacio de kernel 12
2.1.3. Ficheros y Sistemas de Ficheros (*FS*) 15
2.1.4. Espacios de nombres (*Namespaces*) 19
2.1.5. Módulos del Kernel de *Linux* (*LKMs*) 23
2.1.6. Entorno de desarrollo 26
2.2. Descripción del problema 26
2.3. Requisitos 26
2.4. Competencias 26
2.5. Metodología 26
2.6. Plan de trabajo 27

3. Plataforma de desarrollo **28**

4. Diseño **29**
4.1. Snippets 29
4.2. Verbatim 29
4.3. Ecuaciones 30
4.4. Tablas o cuadros 30

5. Conclusiones **32**
5.1. Conclusiones 32

5.2. Corrector ortográfico 33

Índice de figuras

1.1. Principales tipos de *malware* [**?**]. 2

2.1. Árbol genealógico de los sistemas operativos *Unix* [**?**]. 9
2.2. Anillos de privilegios en sistemas operativos modernos [**?**, **?**]. 10
2.3. Estructura de un sistema operativo moderno [**?**]. 11
2.4. Gestión de memoria virtual para procesos en *Linux* [**?**]. 13
2.5. División lógica de memoria en Espacio de Usuario y Espacio del Kernel [**?**]. 14
2.6. Estructura jerárquica estándar de directorios en *Linux* (*FHS*). 16
2.7. Gestión de sistemas de ficheros en *Linux* [**?**]. 18
2.8. Punto de montaje: unificación de múltiples sistemas de ficheros en un único árbol lógico [**?**]. 19
2.9. Tipos de *Namespaces* [**?**]. 21
2.10. Ejemplo de arquitectura *Docker* [**?**]. 23

Listado de códigos

2.1. Ejemplo de salida del comando `ps aux` en un terminal Linux 20
2.2. Comandos básicos de gestión de LKMs 25

4.1. Función para buscar elementos 3D en la imagen 29
4.2. Cómo usar un Slider 30

Listado de ecuaciones

4.1. Ejemplo de ecuación con fracciones 30
4.2. Ejemplo de ecuación con array y letras y símbolos especiales 30

Índice de tablas

4.1. Parámetros intrínsecos de la cámara 31

Capítulo 1
Introducción

«*El mayor enemigo de la seguridad es la complejidad.*»

Bruce Schneier *(A Plea for Simplicity, 1999)* [**?**].

Hoy en día, la seguridad de los sistemas es un aspecto fundamental a tener en cuenta, cobrando cada vez más importancia a medida que alojamos mayor cantidad de información sensible en ellos y se convierten en sistemas mucho más complejos. Por lo tanto, podrían quedar expuestos a un gran número de amenazas si no se protegen debidamente. Como advierte el ensayo mostrado al inicio del capítulo, la inherente complejidad de estas infraestructuras facilita la proliferación de *malware* diseñado para operar e infiltrarse desde la más estricta invisibilidad.

Un sistema operativo moderno no es una estructura o proceso simple, sino un entrelazado conjunto de capas o niveles concéntricos de abstracción que actúan respecto a un sistema de hardware. Se puede describir como un sistema realmente complejo, con una gran cantidad de componentes y funcionalidades. En consecuencia, esta complejidad hace que sea difícil de proteger y monitorizar, requiriendo de un gran conocimiento sobre el mismo para saber cómo actuar ante posibles ataques.

A su vez, por parte del atacante, se requiere de un gran conocimiento técnico para diseñar un ataque efectivo, pero también de una gran creatividad y habilidad para encontrar vulnerabilidades y las formas de explotarlas sin ser detectado, dependiendo de su finalidad.

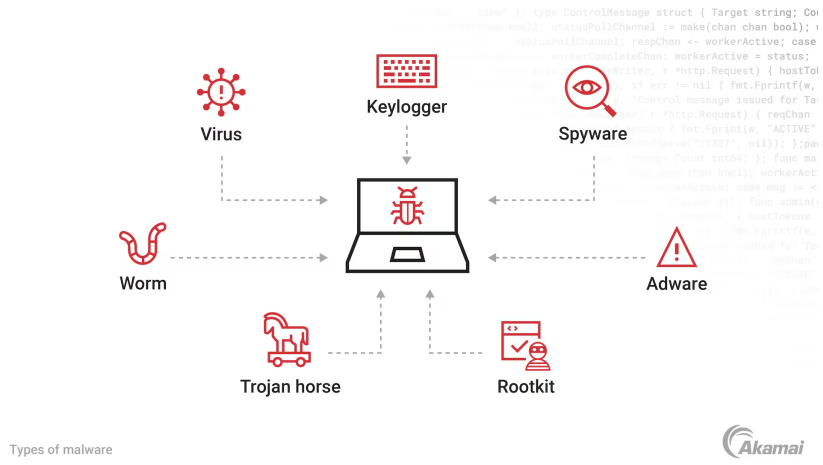
Esto hace de la ciberseguridad una disciplina realmente amplia y compleja, donde atacantes y defensores se encuentran en una constante carrera por encontrar nuevas formas de comprometer o proteger los sistemas.

Este proyecto se centra en el ámbito de la seguridad de los sistemas operativos *Linux*, concretamente en la protección ante amenazas que se infiltran en el núcleo del sistema, como los *rootkits*, evitando la manipulación de información sensible hacia los usuarios mediante un canal de reporte seguro y verificable.

Para comprender en detalle el escenario descrito y contextualizar la temática del proyecto, a continuación se establecerán conceptos básicos sobre la ciberseguridad y sus principales finalidades, especialmente sobre la amenaza principal en la que se centra el proyecto, los *rootkits*.

1.1. Ciberseguridad

En el ámbito de los sistemas de información, la ciberseguridad comprende el conjunto de arquitecturas, tecnologías y prácticas diseñadas para proteger los sistemas, datos y redes frente a ataques que puedan comprometer su integridad, confidencialidad o disponibilidad, ya sean intencionados o no. El principal flujo de estas amenazas se materializa mediante ***software* malicioso** (***malware***), concepto que engloba cualquier código diseñado para infiltrarse en un sistema, dañarlo o robar datos sin el consentimiento de su propietario [**?**].



*Fuente: Elaboración propia basada en "Akamai Technologies", "¿Qué es el malware?" [**?**].*

Figura 1.1: Principales tipos de *malware* [**?**].

Actualmente, existen diversas familias de *malware* clasificadas principalmente según su comportamiento y objetivo. Por un lado, destacan las amenazas orientadas a la

rápida infiltración y propagación, como los ***virus***, ***gusanos*** (*worms*) y ***troyanos***, los cuales aprovechan vulnerabilidades técnicas o el engaño psicológico del usuario para acceder al sistema. Por otro lado, encontramos variantes focalizadas en el espionaje y/o el lucro económico, como el ***ransomware***, que cifra datos críticos o personales exigiendo un posterior rescate económico, y el ***spyware*** junto a los ***keyloggers***, diseñados para vigilar y extraer información sensible del sistema de forma inadvertida.

No obstante, dentro de este ecosistema ofensivo existe una categoría que destaca por su extrema criticidad y que constituye el foco principal de este proyecto: los ***rootkits***. A diferencia del *malware* tradicional, que opera en el *Espacio de Usuario*, los *rootkits* están diseñados específicamente para comprometer las capas más profundas de la máquina en el *Espacio del Kernel*, escalando privilegios y manipulando información del núcleo (*kernel*). En próximos apartados se profundizará concretamente en esta peligrosa amenaza.

Sin embargo, para que este *malware* logre infiltrarse en la infraestructura del sistema, los atacantes necesitan métodos de acceso y mecanismos de persistencia. A su vez, precisan de un correcto análisis, identificación y explotación de las vulnerabilidades del sistema a través de las cuales poder infiltrarse.

1.1.1. Vulnerabilidades y Ataques

En el contexto de la ciberseguridad de sistemas, una **vulnerabilidad** se define como una debilidad en el diseño, implementación o configuración de un sistema que puede ser aprovechada por un agente malicioso para comprometer su integridad, confidencialidad o disponibilidad [**?**].

Estas brechas de seguridad constituyen el principal motor de entrada a intrusos y pueden clasificarse fundamentalmente en tres grandes grupos según su naturaleza:

- **Vulnerabilidades de *software***: Fallos a nivel de código fuente del sistema operativo o en las aplicaciones. Típicos ejemplos pueden ser la falta de sanitización o filtrado de datos, derivando en inyecciones de código; o por otro lado, el desbordamiento de *buffer* (*Buffer Overflow*), pudiendo sobrescribir la memoria

contigua de un proceso.

- **Vulnerabilidades humanas**: Debidas a la falta de concienciación, errores o negligencias de los usuarios propietarios del sistema, pudiendo ser el eslabón más débil de la cadena de seguridad.

- **Vulnerabilidades de configuración**: Diseño o implementaciones inseguras de la propia arquitectura del sistema, como el uso de credenciales por defecto, servicios de red expuestos o una asignación incorrecta de privilegios.

Para aprovechar estas vulnerabilidades y realizar la intrusión del *malware*, los atacantes hacen uso de diversas **herramientas de ataque** y técnicas específicas para cada escenario:

- **Ataques de ingeniería social**: Orientados a explotar las vulnerabilidades humanas. Destacan el ***phishing*** y sus variantes (***spear phishing***), cuyo objetivo es la manipulación psicológica del usuario mediante comunicaciones fraudulentas que permitan revelar credenciales o ejecutar *software* malicioso de forma inadvertida.

- ***Exploits* y *Payloads***: Para atacar mediante fallos de configuración o de *software*, se emplean *exploits*. Estos son fragmentos de código diseñados para aprovechar una debilidad técnica y forzar un comportamiento no previsto en la máquina. Su principal objetivo suele ser inyectar una carga útil detonante (*payload*), como ***backdoors*** o herramientas de administración remota (***RATs***).

- **Ataques de *Privilege Escalation***: Una vez que el atacante logra comprometer mínimamente el sistema, utiliza herramientas ofensivas para aprovechar vulnerabilidades secundarias del mismo y elevar sus permisos hasta incluso obtener el control de administrador (*root*).

En definitiva, independientemente de la vulnerabilidad detectada y la técnica ofensiva empleada, existe una necesidad crítica de implementar mecanismos de defensa robustos y una monitorización de estado segura. Esto, añadido a una adecuada educación y concienciación de los usuarios, resulta fundamental para mitigar los riesgos y poder proteger en gran medida al sistema.

1.1.2. Defensas

Para mitigar y protegerse ante las principales amenazas de este ámbito, se creó el enfoque estratégico basado en el principio de **defensa en profundidad** (***Defense in Depth***). Este concepto consiste en desplegar múltiples capas de seguridad superpuestas, de modo que si un control falla o es evitado, el atacante pueda encontrarse con otro más adelante y complicar el acceso. Principalmente mediante este enfoque, podemos dividir los métodos defensivos en dos categorías [**?**].

En primer lugar, las **medidas preventivas** buscan reducir la superficie de ataque y evitar que la intrusión llegue a materializarse. Entre las prácticas más destacadas se encuentran:

- **Concienciación y capacitación (*Security Awareness*)**: Educar a los usuarios para identificar y reportar ataques de ingeniería social, reduciendo el factor de error humano.
- **Bastionado de sistemas (*Hardening*)**: Consiste en mantener las aplicaciones y el sistema operativo actualizados para mitigar posibles *exploits*. Por otro lado, implica también la configuración segura del sistema, como deshabilitar servicios innecesarios o restringir el tráfico de red mediante el uso de ***firewalls***.
- **Principio de Menor Privilegio (*PoLP*)**: Garantiza que los usuarios, procesos y aplicaciones operen únicamente con los permisos mínimos e indispensables que precisen para cumplir su función. Esto limita drásticamente el impacto de una brecha inicial de acceso y dificulta la escalada de privilegios.

Por otro lado, asumiendo que las barreras preventivas pueden ser superadas, es indispensable implementar mecanismos de **detección, respuesta y monitorización** continua [**?**]:

- **Sistemas de Detección de Intrusos basados en Host (*HIDS*)**: Herramientas que monitorizan la actividad de la máquina, analizan los registros de eventos (*logs*) y detectan alteraciones no autorizadas registradas en ficheros críticos (*File Integrity Monitoring*).
- **Plataformas *EDR* (*Endpoint Detection and Response*)**: Evolución del concepto de antivirus tradicional. En lugar de basarse únicamente en firmas, analizan el comportamiento de los procesos en memoria en tiempo real para poder así detectar y bloquear ejecuciones anómalas.

Cabe destacar que el ámbito de la ciberseguridad se encuentra en constante evolución. La continua sofisticación y mejora de las técnicas ofensivas exige un gran esfuerzo por parte de la industria para investigar y diseñar nuevos métodos de protección. De este modo, los paradigmas de defensa empleados actualmente, como los explicados en este apartado, deben adaptarse de forma dinámica al contexto temporal de cada entorno en el que se utilicen, pudiendo quedar obsoletos y ser ineficaces a corto plazo.

1.1.3. Rootkits

En este contexto de ciberseguridad, un ***rootkit*** se define como un conjunto de fragmentos de *malware* diseñados específicamente para mantener un acceso privilegiado en un sistema, y a su vez, ocultar su presencia y las actividades que prefiera (como procesos, conexiones de red o ficheros) ante los administradores y el *software* de seguridad [**?**].

A diferencia de otras amenazas, su principal objetivo no es propagarse o causar un daño destructivo inmediato, sino lograr una **persistencia indetectable** a largo plazo tras una intrusión inicial en el sistema objetivo. Para alcanzar este nivel de evasión, los *rootkits* alteran el flujo normal de ejecución del sistema operativo mediante técnicas de interceptación y modificación de código, típicamente conocidas como ***hooking***.

Dependiendo de la capa en la que operen y del nivel de privilegios que puedan llegar a escalar, se pueden dividir en dos grandes categorías en sistemas *Linux*:

- ***Rootkits* de Modo Usuario (*User-space*):** Operan en la región de menor privilegio (*Ring 3*). La técnica más común consiste en capturar las funciones de la biblioteca estándar de C (`libc`) mediante mecanismos como la carga dinámica de bibliotecas a través de la variable `LD_PRELOAD`. De esta forma, logran interceptar las peticiones a dichas funciones (*hooks*) como `open` o `read`, antes de llegar al *kernel*, ocultando y filtrando los resultados en la salida hacia el usuario. Investigaciones recientes demuestran que las herramientas de detección clásicas que operan en el nivel de privilegios de usuario normal resultan fácilmente evadibles, generando una falsa sensación de seguridad [**?**].
- ***Rootkits* de Modo Kernel (*Kernel-space*):** Representan la amenaza más

crítica, ya que operan en el nivel de máximo privilegio (*Ring 0*). Suelen implementarse mediante Módulos del Kernel (*LKMs*). Al residir en el *kernel* del sistema, adquieren privilegios de administrador sobre las estructuras de datos críticos y la Tabla de Llamadas al Sistema (*sys_call_table*). Esto les permite manipular el sistema desde dentro, volviendo completamente inútiles las herramientas de monitorización que operan por encima de ellos con menos privilegios.

El desafío fundamental que plantean estos tipos de amenazas radica en un axioma crítico en ciberseguridad: ***un sistema comprometido no puede proporcionar información fiable sobre sí mismo***. Con base en estos argumentos, intentar detectar o mitigar la acción de un *rootkit* utilizando mecanismos de seguridad que operan en un nivel de privilegios inferior al del atacante es una estrategia totalmente ineficaz. Por esta razón, se precisa diseñar arquitecturas defensivas y de monitorización robustas en las capas más profundas del sistema, con el fin de garantizar la integridad de los datos y el correcto funcionamiento del entorno.

Tomando como referencia el complejo panorama de la ciberseguridad descrito y la gran amenaza que suponen los ***rootkits***, el presente proyecto propone el diseño y desarrollo de un sistema de reporte seguro para entornos *Linux*, enfocado principalmente en la protección frente a este tipo de *malware*. Para comprender en profundidad el entorno en el que se desenvuelven estas amenazas, el siguiente capítulo establecerá el marco tecnológico de los sistemas operativos *Linux* actuales, detallando su diseño, arquitectura y funcionamiento. Posteriormente, se abordarán los objetivos y metodologías adoptadas para el diseño, integración y validación de este sistema de reporte seguro.

Capítulo 2
Estado del Arte

Para poder diseñar e implementar una solución de seguridad efectiva frente a amenazas *rootkits*, es necesario comprender el entorno tecnológico en el que se trabaja. Por ello, en el presente capítulo se analizará en primer lugar la arquitectura y estructura de los sistemas operativos *Linux*, detallando sus mecanismos internos, como su gestión de ficheros, memoria y modularidad.

2.1. Sistemas operativos *Linux*

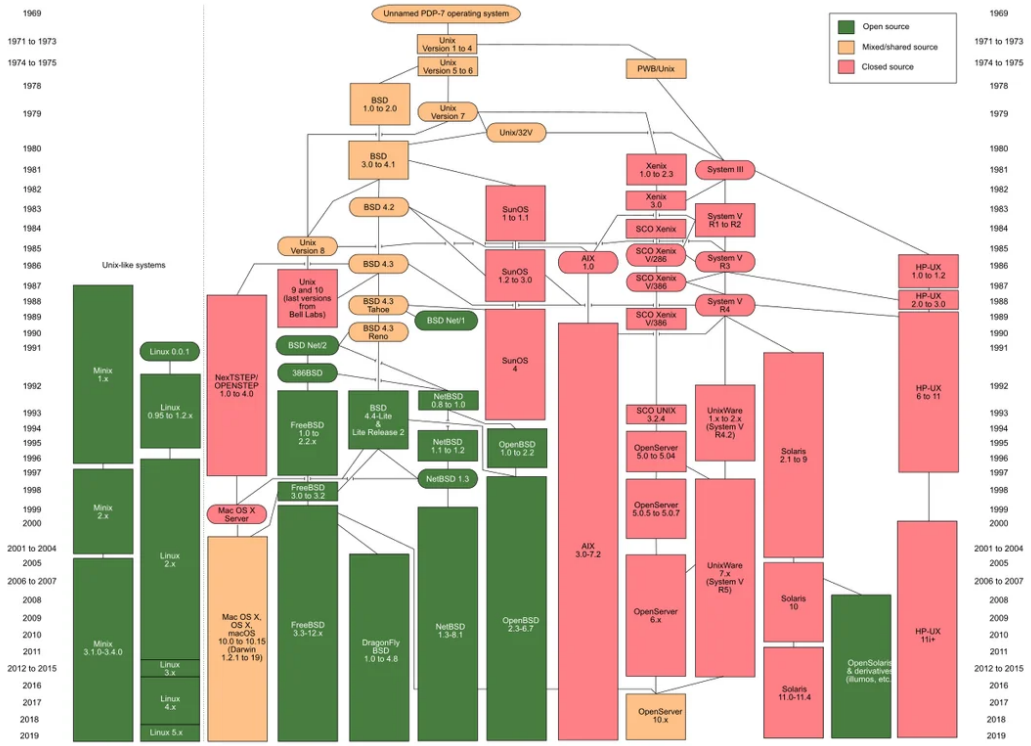
No existe una definición canónica y universal para el concepto de sistema operativo, tratándose de una idea hasta cierto punto relativa y subjetiva dependiendo de su cometido y contexto de uso [**?**].

Desde un punto de vista físico, puede definirse como el conjunto de programas que permite utilizar la máquina como sistema, haciendo lo necesario para que puedan ejecutarse múltiples procesos sobre el mismo *hardware*, multiplexando la máquina en tiempo (administrando uso de *CPU*) y espacio (gestionando la memoria *RAM*).

Por otro lado, desde un punto de vista lógico, se puede definir como el programa o conjunto de los mismos que actúan como intermediarios entre el usuario y el *hardware*, proporcionando una interfaz como abstracción de procesos e instrucciones internas, para poder interactuar con la máquina y sus recursos de manera eficiente y robusta.

En el mercado tecnológico actual existen diferentes sistemas operativos de propósito general, destacando opciones comerciales de código cerrado como *Microsoft Windows* o *macOS*, orientados generalmente a ordenadores, y por otra parte, *Android* e *iOS*, principalmente diseñados para dispositivos móviles. Sin embargo, dentro del ámbito

de la administración de infraestructuras, ciberseguridad y desarrollo de *software*, *GNU/Linux* (de la familia *Unix*) se considera como el estándar de referencia absoluto.



Fuente: "Anthes, Gary", "Unix turns 40: The past, present and future of a revolutionary OS" (2009) [?].

Figura 2.1: Árbol genealógico de los sistemas operativos *Unix* [?].

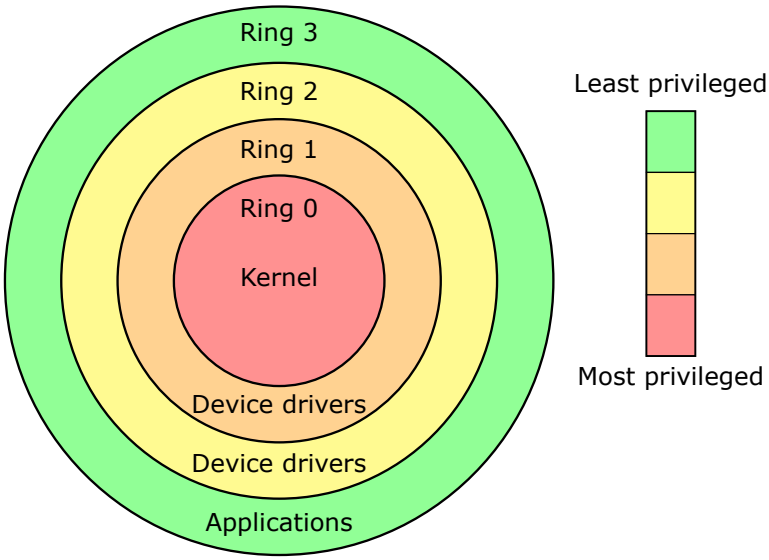
Como ilustra la imagen anterior, *Unix* es el sistema pionero creado en los años 70 que sentó las bases de la informática e inspiró directamente el desarrollo de *Linux*, considerado como su evolución natural de código abierto. También *macOS* proviene de esta familia; sin embargo, este sistema operativo ha evolucionado bajo un modelo comercial semi-privativo y de código parcialmente cerrado.

La hegemonía de *GNU/Linux* en estos sectores se debe principalmente a su filosofía de código abierto (*open-source*). Disponer del código fuente de manera pública, tanto directamente en los directorios del sistema (típicamente en `/usr/src`) como en repositorios públicos en línea como *GitHub*, permite a una gran comunidad mundial de desarrolladores auditar, depurar y mejorar el código del mismo de forma continua. Esta transparencia garantiza un nivel de adaptabilidad y fiabilidad muy superior al de los sistemas comerciales, permitiendo una configuración del entorno a medida según las necesidades requeridas.

Por estos motivos, se considera *GNU/Linux* el sistema operativo óptimo para desarrollar este proyecto. Por consiguiente, a partir de este punto se asumirá por defecto que nos encontramos en un entorno *Linux*.

2.1.1. Arquitectura y anillos de privilegios

Para garantizar su estabilidad y seguridad, los sistemas operativos modernos no confían únicamente en restricciones lógicas de *software*, sino que se apoyan de forma entrelazada en mecanismos de protección implementados a nivel de *hardware* por el propio procesador. En arquitecturas de sistemas predominantes como la familia x86, este mecanismo de aislamiento se implementa típicamente a través de dominios concéntricos de protección jerárquicos, comúnmente conocidos como anillos de protección (*Protection Rings*) [**?**].



*Fuente: "Colaboradores de Wikipedia", "Anillo (seguridad informática)" (2026) [**?**].*

Figura 2.2: Anillos de privilegios en sistemas operativos modernos.

Esta arquitectura determina principalmente cuatro niveles de privilegios concéntricos, comúnmente llamados "anillos", numerados del 0 al 3. El nivel de privilegio implica qué instrucciones de la CPU puede ejecutar un proceso y a qué regiones de la memoria puede acceder al ejecutarse, distinguiendo estos cuatro anillos [**?**]:

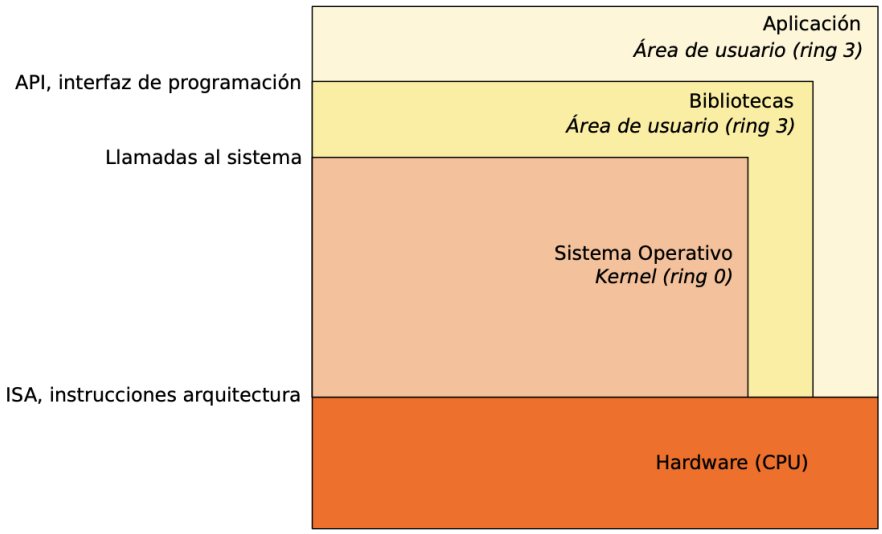
- ***Ring 0*:** Nivel de máximo privilegio, donde reside típicamente el núcleo (*kernel*).

Los programas ejecutados en este anillo tienen prácticamente acceso directo e irrestricto a todo el *hardware*, la memoria y las instrucciones del procesador.

- ***Ring 1 y 2*:** Originalmente diseñados para servicio y control del propio sistema con privilegios intermedios, aunque en la práctica han sido mayormente descartados por la mayoría de sistemas operativos, debido a su complejidad y bajo rendimiento.

- ***Ring 3*:** Nivel de menor privilegio. Los procesos lanzados en este anillo no pueden interactuar directamente con el *hardware*, debiendo delegar cualquier operación crítica a niveles de mayor privilegio.

Por motivos de eficiencia y portabilidad a otras arquitecturas de procesadores (como *ARM*, que normalmente no posee tantos niveles), *Linux* simplifica este modelo y utiliza exclusivamente dos de estos anillos. Por un lado, se aísla el *kernel* (núcleo del sistema), ejecutándolo íntegramente en el *Ring 0*, y por otro lado, se confina el resto de programas de usuario en la región del *Ring 3* [**?**, **?**].



Fuente: "Guardiola Múzquiz, Gorka and Soriano Salvador, Enrique", "Fundamentos de Sistemas Operativos: Una aproximación práctica usando Linux" (2024) [?].

Figura 2.3: Estructura de un sistema operativo moderno.

Básicamente, la estructura del sistema operativo se puede describir como una división física de privilegios que se traduce directamente en una arquitectura lógica estructurada en capas construidas unas encima de otras [**?**].

En la base se encuentra el *hardware* físico de la máquina, con el cual el núcleo interactúa directamente a través de las instrucciones de la arquitectura (ISA) disponibles. Inmediatamente por encima, operando con privilegios absolutos sobre el sistema, en el *Ring 0*, reside el *kernel*. Este es el corazón del sistema operativo, responsable de gestionar los recursos de la máquina y de actuar como puente fundamental entre el *hardware* y las aplicaciones de usuario. Posteriormente, en las capas más externas del mismo, se encuentran las bibliotecas del sistema, encargadas de proporcionar la ***interfaz de programación de aplicaciones (API)***, y las aplicaciones finales, las cuales se encuentran confinadas en el área de usuario (*Ring 3*).

Debido a esta separación, si algún proceso de esta región de usuario requiere realizar una acción privilegiada (como leer un archivo del disco, enviar un paquete de red o reservar memoria), no puede acceder al *hardware* directamente por sí mismo. En su lugar, debe solicitar dicha operación privilegiada al *kernel* atravesando la frontera del *Ring 0* mediante un mecanismo de comunicación estandarizado y controlado, comúnmente conocido como ***llamadas al sistema*** (*System Calls* o *syscalls*).

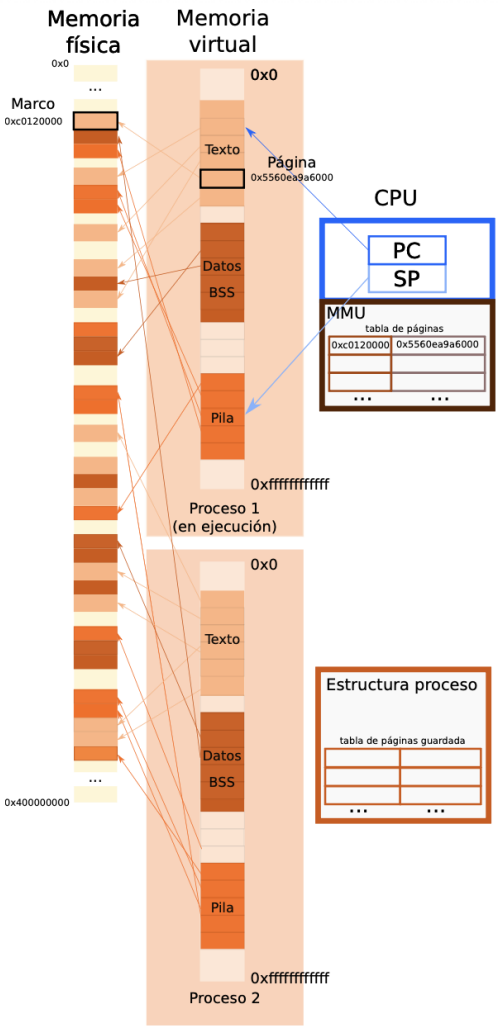
Esta barrera arquitectónica no solo define qué instrucciones se pueden ejecutar y dónde, sino que obliga al sistema operativo a dividir la memoria principal de rápido acceso (*RAM*) en dos grandes regiones virtuales completamente aisladas en ***espacio de usuario*** y ***espacio de kernel***, garantizando así controlar que ningún proceso de usuario pueda corromper datos sensibles internos del sistema [**?**, **?**].

2.1.2. Memoria: Espacio de usuario y espacio de kernel

Para comprender cómo se materializa la división de privilegios descrita en el apartado anterior realmente en la memoria física de la máquina (*RAM*), es necesario introducir el concepto de *memoria virtual*.

En los sistemas *Linux* modernos, los procesos no acceden directamente a las direcciones físicas de la memoria *RAM*. En su lugar, el *kernel* y la unidad de gestión de memoria (*MMU*) de la *CPU* proporcionan a cada proceso la ilusión de tener espacio exclusivo y contiguo a través de la ***memoria virtual***. Para lograrlo, la *MMU* traduce dinámicamente las direcciones virtuales de memoria a ubicaciones físicas reales y viceversa, consultando una ***tabla de páginas***. De esta forma, se garantiza que

múltiples procesos puedan ejecutarse simultáneamente de forma aislada y segura, cada uno con su propio mapa de direcciones gestionado íntegramente por el *kernel* [**?**].



*Fuente: "Guardiola Múzquiz, Gorka and Soriano Salvador, Enrique", "Fundamentos de Sistemas Operativos: Una aproximación práctica usando Linux" (2024) [**?**].*

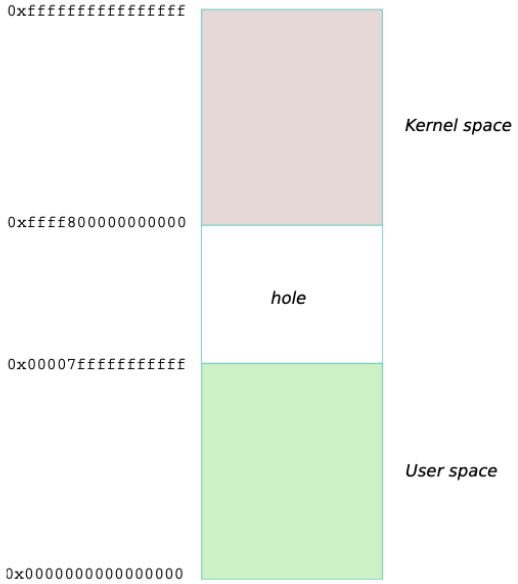
Figura 2.4: Gestión de memoria virtual para procesos en *Linux*.

A su vez, este espacio de *memoria virtual* se divide de forma estricta en dos grandes regiones, fuertemente relacionadas con la arquitectura de anillos de protección del procesador:

- **Espacio de Usuario (*User Space*):** Es la porción inferior de la memoria virtual, asociada al *Ring 3*. En esta región se cargan y ejecutan las aplicaciones de usuario, sus bibliotecas y los datos pertinentes. Si un proceso comete un error "crítico" en esta región (como intentar acceder a memoria que no le pertenece),

el sistema operativo interviene para finalizar dicho proceso y genera un error (típicamente conocido como *Segmentation Fault*), evitando que el fallo afecte al resto del sistema.

- **Espacio del Kernel (*Kernel Space*):** Es la porción superior de la memoria virtual, reservada exclusivamente para el ***kernel*** con sus módulos y los controladores de dispositivos (*drivers*). Al estar asociado al *Ring 0*, el código que reside aquí tiene acceso total a la memoria física y, por consiguiente, al *hardware*. Un fallo de programación o una intrusión maliciosa en este nivel compromete todo el sistema, pudiendo provocar una caída total del mismo (*Kernel Panic*).



Fuente: "Guardiola Múzquiz, Gorka and Soriano Salvador, Enrique", "Fundamentos de Sistemas Operativos: Una aproximación práctica usando Linux" (2024) [?].

Figura 2.5: División lógica de memoria en Espacio de Usuario y Espacio del Kernel.

De la misma manera que la *memoria virtual* abstrae la complejidad de organización de la memoria física (*RAM*) para garantizar el aislamiento y la ejecución segura de los procesos, el sistema operativo proporciona a su vez una abstracción equivalente para la gestión del almacenamiento persistente y la interacción con el resto de componentes del sistema. Esta importante tarea recae sobre otro gran pilar estructural de un sistema operativo, en este caso *Linux*: su *sistema de ficheros* (*file system*).

2.1.3. Ficheros y Sistemas de Ficheros (*FS*)

Una de las características más destacadas de la arquitectura de *Linux*, heredada directamente de la familia original de *Unix*, es el principio fundamental de que «todo es un fichero». En este tipo de sistemas operativos, no solo los documentos de texto o los programas ejecutables binarios son tratados como ficheros, sino también los directorios, las conexiones de red (*sockets*), los dispositivos de *hardware* (como terminales o discos duros), así como la información dinámica de los procesos en ejecución y del propio sistema (como los ficheros virtuales situados en el directorio `/proc`) [**?**].

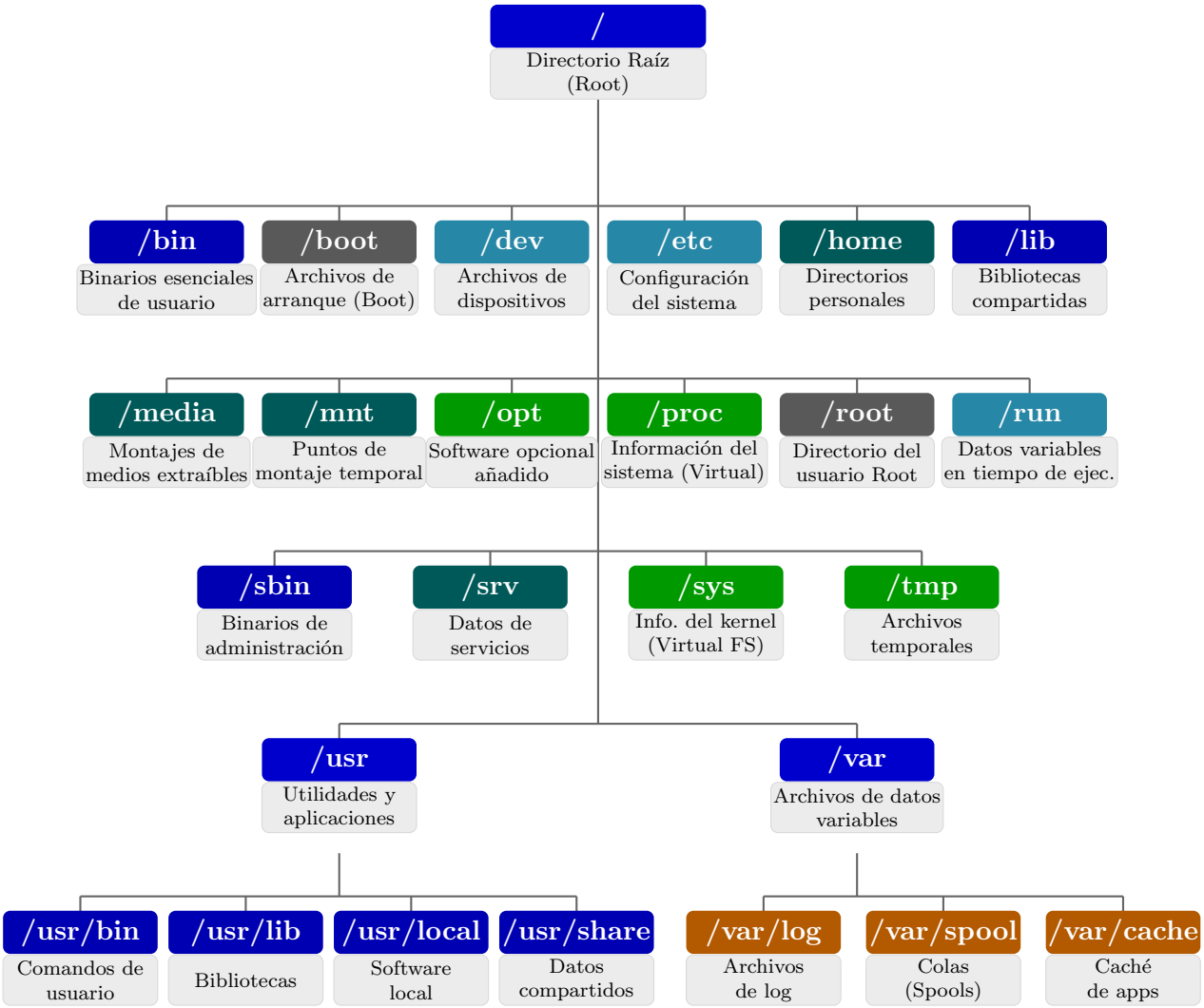
Para estructurar, acceder y organizar toda esta enorme cantidad de información, el sistema operativo emplea un **Sistema de Ficheros (*FS*)**, el cual se puede definir como el conjunto de reglas, métodos y jerarquías lógicas encargados de gestionar cómo los ficheros se nombran, se almacenan y se recuperan en un medio lógico.

Debido a la gran cantidad de datos y recursos tanto físicos como lógicos que maneja la máquina, es necesario mantener un orden universal de los mismos. El sistema operativo necesita saber exactamente dónde ubicar cada uno de los diferentes tipos de ficheros del sistema.

Como solución a esta necesidad organizativa, se establece un estándar jerárquico estructural de directorios de todo el sistema para facilitar la localización de los recursos disponibles dependiendo de su finalidad y alcance.

Como se muestra en la Figura 2.6, el estándar *FHS* clasifica a los ficheros según su propósito y alcance en un sistema operativo *Linux* moderno. Entre los directorios más relevantes e importantes, se encuentran los siguientes [**?**, **?**]:

- `/bin` y `/sbin`: Contienen los programas y ejecutables binarios esenciales para el usuario y para el administrador del sistema, respectivamente.
- `/etc`: Almacena los ficheros de configuración globales de la máquina y de las aplicaciones instaladas.
- `/dev`: Refleja el principio de «todo es un fichero», alojando ficheros especiales que representan directamente a los dispositivos de *hardware* (como terminales o discos `/dev/sda`).



Fuente: Elaboración propia basada en "Albritton, William McDaniel", ""Module 2, Session 7 - Linux File System Structure"" (2016) [?].

Figura 2.6: Estructura jerárquica estándar de directorios en *Linux* (*FHS*).

- `/var`: Almacena ficheros de datos variables cuyo tamaño y contenido cambian durante el funcionamiento del sistema. Su subdirectorio `/var/log` es vital, pues contiene los registros de eventos (*logs*) de toda la máquina.

- `/proc` y `/sys`: Son pseudo-sistemas de ficheros virtuales creados dinámicamente en la memoria (*RAM*) por el *kernel*, no se almacena nada en el disco físico. Exponen información sensible en tiempo real sobre los procesos en ejecución de la máquina, datos de configuración y reportes de estado del propio *kernel*.

- `/boot`: Aloja los archivos estáticos necesarios para el proceso de arranque e inicialización del sistema operativo, incluyendo el propio *kernel* compilado (la imagen *vmlinuz*) y la configuración del gestor de arranque.

- `/tmp`: Almacena ficheros temporales creados por el usuario o las aplicaciones en ejecución. Por motivos de limpieza y rendimiento, su contenido suele borrarse automáticamente cada vez que se apaga o reinicia la máquina.

- `/lib`: Contiene las bibliotecas necesarias para ejecutar los binarios de `/bin` y `/sbin`. Por otro lado, es una carpeta crítica ya que aloja los módulos dinámicos del *kernel* (*LKMs*).

- `/home` y `/root`: Alojan los directorios y configuraciones de trabajo de los usuarios y del administrador o superusuario del sistema, respectivamente.

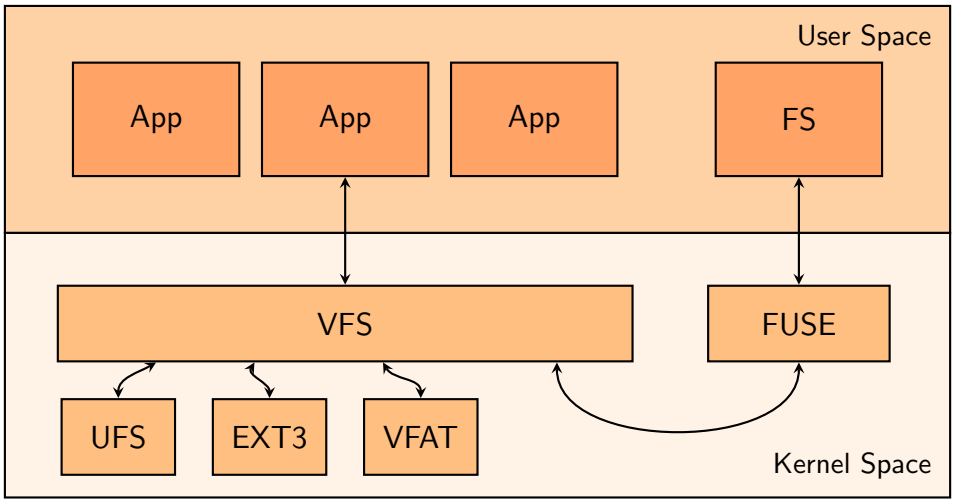
- `/usr`: Es uno de los directorios más complejos y grandes. Contiene aplicaciones, documentación, bibliotecas y ejecutables que no son vitales para el sistema, pero sí para el uso general y la experiencia del usuario.

- `/mnt` y `/media`: Actúan como los **puntos de montaje** estándar del sistema, mejor explicados más adelante. `/mnt` suele ser utilizado para montajes temporales realizados manualmente por el administrador, mientras que `/media` suele utilizarse por el sistema para montar automáticamente medios extraíbles (como unidades de memoria *USB*).

- `/opt`: Se reserva para la instalación de paquetes opcionales, privativos o de terceros que no forman parte de los repositorios oficiales de la distribución, manteniéndolos aislados del resto del sistema operativo.

- `/run` y `/srv`: `/run` almacena datos volátiles en memoria sobre el estado de los servicios desde el último arranque (como los *PIDs* de los procesos), mientras que `/srv` contiene datos específicos ofrecidos por la máquina (como los archivos de un servidor web o *FTP*).

Para gestionar esta gran heterogeneidad de recursos y permitir la coexistencia de múltiples sistemas de ficheros físicos (como *ext4* o *NTFS*) en un mismo sistema anfitrión, el *kernel* implementa una capa de abstracción denominada **Sistema de Ficheros Virtual (*VFS*)**, situada entre la capa de llamadas al sistema (*syscalls*) y los *drivers* de bajo nivel. Su función es proporcionar una *API* unificada que permite a las aplicaciones del *Espacio de Usuario* utilizar llamadas estándar (como `read()` o `write()`) sobre cualquier fichero o dispositivo, abstrayendo por completo los detalles del *hardware* de la máquina [**?**].



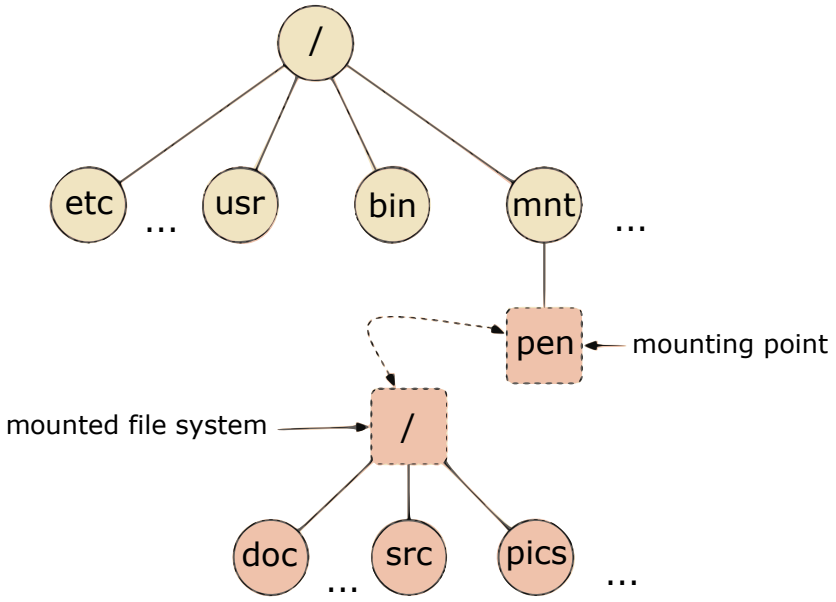
*Fuente: "Guardiola Múzquiz, Gorka and Soriano Salvador, Enrique", "Fundamentos de Sistemas Operativos: Una aproximación práctica usando Linux" (2024) [**?**].*

Figura 2.7: Gestión de sistemas de ficheros en *Linux*.

Principalmente, el *VFS* actúa como un enrutador central entre el *Espacio de Usuario* y el *Espacio del Kernel*, recibiendo las peticiones de las aplicaciones y redirigiéndolas de forma transparente al *driver* físico correspondiente. Además, los sistemas modernos soportan mecanismos avanzados como *FUSE* (*Filesystem in Userspace*), que permite al *VFS* delegar la gestión de ciertos sistemas de ficheros a procesos que se ejecutan de forma segura directamente en el *Espacio de Usuario*.

Para integrar distintos sistemas de ficheros dentro del sistema anfitrión, *Linux* prescinde de unidades aisladas y utiliza la tecnología de **montaje** (*mounting*). Este mecanismo unificador "engancha" lógicamente cualquier sistema de ficheros externo a un directorio existente (típicamente `/mnt` o `/media`), considerado **punto de montaje** (*mount point*). De este modo, se crea una estructura de directorios continua y transparente que abstrae al usuario de la naturaleza del medio externo subyacente [**?**].

Como se ha expuesto en los apartados anteriores, la arquitectura de *Linux* se fundamenta en recursos fuertemente centralizados: un único *kernel*, memoria física compartida, una jerarquía de directorios global, entre otros. Para romper con esta centralización y dotar al sistema de capacidades de aislamiento, el *kernel* implementa la tecnología de los *Namespaces*, la cual permite el confinamiento de procesos y recursos, tanto físicos como lógicos, dentro de un entorno aislado y controlado. Este concepto se explica en detalle en el siguiente apartado [**?**, **?**].



*Fuente: "Guardiola Múzquiz, Gorka and Soriano Salvador, Enrique", "Fundamentos de Sistemas Operativos: Una aproximación práctica usando Linux" (2024) [**?**].*

Figura 2.8: Punto de montaje: unificación de múltiples sistemas de ficheros en un único árbol lógico.

2.1.4. Espacios de nombres (*Namespaces*)

Para poder explicar en profundidad cómo funcionan los *Namespaces*, es necesario definir previamente cómo gestiona y clasifica el sistema operativo los programas que se ejecutan en la máquina.

A todo programa que se encuentre en ejecución y alojado en la memoria virtual se le denomina ***proceso***. Para administrar de forma eficiente la gran cantidad de procesos que pueden coexistir simultáneamente en el sistema, el *kernel* asigna a cada uno de ellos una serie de atributos e identificadores críticos, considerados de gran relevancia en cuanto al registro del estado del sistema:

- ***PID* (*Process ID*):** Es un número entero único asignado dinámicamente a cada proceso activo que lo identifica de forma inequívoca dentro del sistema.
- ***UID* (*User ID*):** Es el identificador numérico que asocia el proceso a un usuario concreto de la máquina. Este atributo permite dictaminar los permisos y privilegios principales que tiene el proceso para interactuar con el sistema de ficheros, enviar señales o ejecutar ciertas llamadas al sistema (*syscalls*).

- ***GID* (*Group ID*):** Es el identificador numérico del grupo al que pertenece el proceso. Se utiliza de forma complementaria al *UID* para determinar y refinar los permisos de acceso a recursos compartidos del sistema.

- ***Comando*:** Es el nombre o referencia directa del fichero ejecutable binario que dio origen al proceso (por ejemplo, `/bin/ls` o `/usr/bin/python3`).

```
$localhost:$ ps aux
USER        PID %CPU %MEM  VSZ   RSS TTY     STAT START  TIME COMMAND
root          1 0.7 0.3 22208 12444 ?       Ss   12:37  0:04 /sbin/init
root          2 0.0 0.0     0     0 ?       S    12:37  0:00 [kthreadd]
root          3 0.0 0.0     0     0 ?       S    12:37  0:00 [pool_workque
root          4 0.0 0.0     0     0 ?       I<   12:37  0:00 [kworker/R-kv
root          5 0.0 0.0     0     0 ?       I<   12:37  0:00 [kworker/R-rc
root          7 0.0 0.0     0     0 ?       I<   12:37  0:00 [kworker/R-sl
root          8 0.0 0.0     0     0 ?       I<   12:37  0:00 [kworker/R-ne
root         11 0.1 0.0     0     0 ?       I    12:37  0:00 [kworker/0:1-
...
noel       3039 53.0 14.7 1217632252 573236 ? Sl 12:39  3:52 /usr/share/co
noel       3145 0.7 2.0 1215834136 78112 ? Sl   12:40  0:02 /usr/share/co
noel       3206 7.9 0.8 82304 31816 ?      Sl   12:41  0:23 /home/noel/.v
root       3327 0.0 0.0     0     0 ?       I<   12:42  0:00 [kworker/0:0H
root       3330 0.2 0.0  9760 3676 pts/0   S    12:42  0:00 su
root       3332 0.8 0.1 14356 7184 pts/0   S    12:42  0:01 zsh
root       3619 0.0 0.0     0     0 ?       I    12:44  0:00 [kworker/u20:
noel       3668 3.1 0.5 4260844 21540 ?    Sl   12:45  0:02 /home/noel/.v
root       3683 0.7 0.0     0     0 ?       I    12:45  0:00 [kworker/u16:
root       3726 700 0.1 10852 4264 pts/0   R+   12:46  0:00 ps aux
```

Código 2.1: Ejemplo de salida del comando `ps aux` en un terminal *Linux* mostrando el listado de procesos actuales en ejecución del sistema.

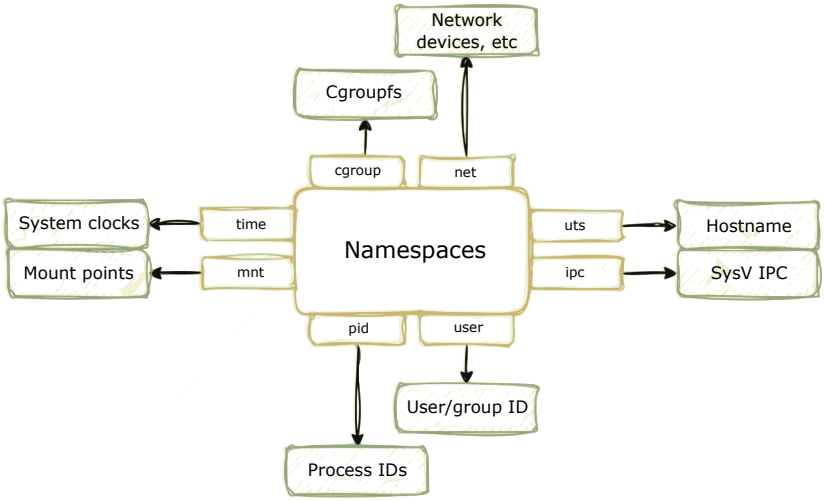
Si no consideramos de momento el concepto de *Namespace*, se puede asumir que todos estos recursos y atributos son de carácter **global**. Esto significa que si, por ejemplo, un proceso tiene asignado el *PID* 3039, ese identificador es único, absoluto y visible para cualquier otra herramienta de monitorización de la máquina (como los ficheros virtuales de `/proc`). Del mismo modo, el usuario *root* (cuyo *UID* es 0) tiene privilegios absolutos sin restricciones sobre todos los procesos del sistema, a diferencia de un usuario estándar con un *UID* distinto (como el 1000), el cual estará mayormente restringido y solo podrá gestionar los procesos que él mismo haya iniciado.

Es en este punto donde la tecnología de *Namespaces* entra en escena, alterando por completo este paradigma. Un ***Namespace*** funciona como una capa de abstracción

lógica que envuelve un recurso global del sistema y lo expone a un grupo de procesos como si fuera un recurso aislado y privilegiado, siendo el *kernel* el encargado de traducir estas identidades, atributos, permisos y recursos de forma totalmente transparente.

En la práctica, esto implica que un proceso ejecutándose dentro de un *Namespace* aislado puede tener asignado el *PID* 1 (el cual típicamente corresponde al primer proceso ejecutado en la máquina, *systemd* o *init*), dándole la ilusión de que es el núcleo central del sistema. Sin embargo, desde la perspectiva exterior del *kernel*, ese mismo proceso es simplemente un proceso regular más con el *PID* 4500, por ejemplo.

Para lograr este nivel de flexibilidad, el *kernel* no aplica un aislamiento monolítico sobre todo el sistema operativo de una única vez. En su lugar, proporciona una arquitectura modular basada en distintos **tipos de *Namespaces***. Cada tipo se encarga de aislar un recurso global específico, permitiendo a los administradores (o a los atacantes) crear combinaciones de los mismos de forma independiente según la finalidad de su uso [**?**].



*Fuente: "Hackerstack and Gmkziz", "Understanding Linux Namespaces" (2024) [**?**].*

Figura 2.9: Tipos de *Namespaces*.

Estos son los principales tipos de *Namespaces* a implementar en un sistema *Linux* moderno:

- ***Mount*** **(`mnt`):** Aísla los puntos de montaje del sistema de ficheros (concepto directamente relacionado con la Figura 2.8). Los procesos dentro de este

Namespace poseen su propia visión de la jerarquía de directorios (*FHS*). Esto implica que montar o desmontar una unidad extraíble (como un dispositivo *USB*) en este espacio aislado no afectará ni será visible para el resto del sistema anfitrión.

- ***PID*** **(`pid`):** Proporciona una jerarquía de procesos independiente. Como se puso de ejemplo antes, permite que el primer proceso ejecutado en ese espacio reciba el *PID* 1, aislando por completo la visibilidad hacia fuera del *Namespace*. De esta manera, se impide que los procesos internos a este espacio puedan interactuar con los procesos externos al mismo.

- ***User*** **(`user`):** Aísla los identificadores de usuario (*UID*) y de grupo (*GID*). En este espacio se permite que un proceso tenga privilegios absolutos de *root* (*UID* 0) dentro del mismo, mientras que en el exterior siga siendo tratado y restringido como un usuario estándar sin privilegios.

- ***Network*** **(`net`):** Virtualiza y aísla toda la pila de red del sistema. Cada *Namespace* de este tipo posee sus propias interfaces de red (incluyendo su propio *loopback*), puertos, direcciones *IP*, reglas de *firewall* y tablas de enrutamiento (*iptables* o *nftables*).

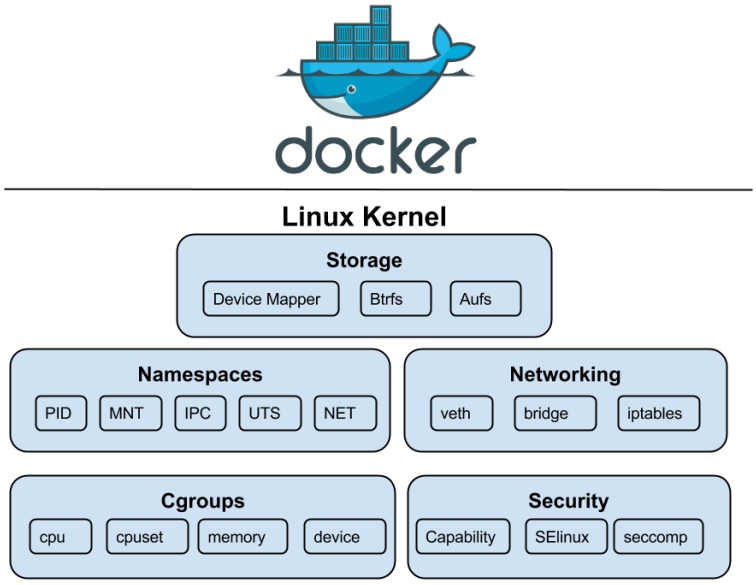
- ***UTS*** **(`uts`):** Permite que un grupo de procesos tenga su propio nombre de máquina (*hostname*) y nombre de dominio *NIS*, de forma independiente al nombre real del servidor físico de la máquina.

- ***IPC*** **(`ipc`):** Aísla los recursos de comunicación entre procesos, como las colas de mensajes (*POSIX*), los segmentos de memoria compartida y los semáforos. Impide que un proceso de este espacio interfiera o intercepte los mensajes de procesos externos.

- ***Cgroup*** **(`cgroup`):** Encapsula la vista del directorio raíz de los grupos de control (*Control Groups*). Esto oculta la jerarquía de políticas y límites de recursos (como *CPU* o memoria) establecidos por el *kernel*, evitando que un proceso aislado los conozca o intente modificarlos.

- ***Time*** **(`time`):** Aísla los relojes internos del sistema, especialmente el reloj de arranque (*boot clock*) y el reloj monotónico (*monotonic clock*). Esto permite que este espacio aislado tenga su propia línea temporal, lo cual es muy útil para migrar contenedores entre diferentes servidores físicos sin alterar su percepción del tiempo de actividad (*uptime*).

La combinación estratégica de estos bloques es, precisamente, el motor tecnológico que hace posible la existencia de tecnologías de virtualización modernas como los contenedores (*Docker*), cuya arquitectura típica se ilustra en la siguiente Figura 2.10.



*Fuente: "Parmar, Kelvin", "Deep Dive into Linux Namespaces: Building Your Own Containers" (2024) [**?**].*

Figura 2.10: Ejemplo de arquitectura *Docker*.

Todas estas tecnologías de aislamiento y abstracción, desde la gestión de memoria de la propia máquina hasta los *Namespaces*, residen y se gestionan en el núcleo del sistema operativo. Sin embargo, pese a su robustez, el *kernel* no es una entidad monolítica e inalterable. Para dotarlo de una alta flexibilidad y permitir la extensión de sus funcionalidades en tiempo real (o la alteración maliciosa de su comportamiento), este dispone de una arquitectura dinámica que permite inyectar y cargar código con máximos privilegios, como se explicará en el siguiente apartado.

2.1.5. Módulos del Kernel de *Linux* (*LKMs*)

Un **Módulo del Kernel** (*LKM*) es código objeto compilado (cuyo código fuente está escrito en lenguaje *C*) que puede ser cargado y descargado del espacio de memoria del núcleo (*kernel*) en tiempo de ejecución. Estos módulos permiten al *kernel* extender sus funcionalidades (como incorporar nuevos sistemas de ficheros) sin necesidad de reiniciar el sistema ni recompilar el *kernel* completo [**?**].

El desarrollo de *LKMs* presenta diferencias drásticas frente a la programación convencional en *Espacio de Usuario*. Al operar directamente en el nivel de máximos privilegios, los módulos no pueden acceder a la biblioteca estándar de *C* (`libc`) y, por consiguiente, a funciones típicas como `printf()` o `malloc()`. Además de esto, al formar parte del propio *kernel*, tampoco pueden utilizar llamadas al sistema (*syscalls*) a través de la interfaz estándar, sino que invocan directamente las funciones internas del *kernel*.

Por otro lado, no existe una salida estándar por consola como ocurre en *Espacio de Usuario*, y para suplirla, el *kernel* proporciona su propia macro de impresión de *logs*: `printk()`. Las trazas generadas mediante esta función se almacenan en un *buffer* circular interno y se registran en ficheros, típicamente en `/var/log/kern.log`. Asimismo, `printk()` exige clasificar cada traza con un nivel de prioridad (*log level*), abarcando eventos críticos o de error (`KERN_EMERG` y `KERN_ERR`), como también informativos o de depuración (`KERN_INFO` y `KERN_DEBUG`).

En cuanto a su compilación, se requieren las cabeceras del *kernel* (paquetes `linux-headers`) que coincidan con la versión exacta del sistema. Mediante un fichero `Makefile`, el código se compila generando un fichero objeto especial empaquetado con extensión `.ko` (*Kernel Object*). Una vez compilado, la gestión de su ciclo de vida se resume en cuatro fases básicas:

- **`insmod`:** Inserción del código en *Espacio de kernel*.
- **`modinfo`:** Reporte de los metadatos y dependencias del módulo.
- **`dmesg`:** Inspección de las trazas de ejecución del *kernel*.
- **`rmmod`:** Extracción del módulo y liberación de los recursos de memoria.

A nivel de código fuente, la estructura lógica de cualquier *LKM* se asienta sobre dos funciones principales. En primer lugar, una **función de inicialización o de carga** (`module_init()`), la cual se ejecuta automáticamente al insertar el módulo en el *kernel* (mediante `insmod`) y se encarga de reservar los recursos necesarios o crear los ficheros virtuales implicados. En segundo lugar, una **función de limpieza o de descarga** (`module_exit()`), invocada al eliminar el módulo cargado (`rmmod`) con el fin de liberar la memoria utilizada y evitar fugas o estados inconsistentes en el sistema.

```
# 0. Compilado del modulo mediante el fichero Makefile
$localhost:$ make
make -C /lib/modules/6.12.41-current-bcm2711/build M=/home/lkms modules
make[1]: Entering directory '/usr/src/linux-headers-6.12.41-current-bcm2711'
  CC [M] /home/lkms/hw.o
  MODPOST /home/lkms/Module.symvers
  CC [M] /home/lkms/hw.mod.o
  LD [M] /home/lkms/hw.ko
make[1]: Leaving directory '/usr/src/linux-headers-6.12.41-current-bcm2711'

# 1. Cargar el modulo (.ko) en el kernel
$localhost:$ sudo insmod hw.ko

# 2. Inspeccionar los metadatos del modulo
$localhost:$ modinfo hw.ko
filename:       /home/lkms/hw.ko
version:        0.1
description:    A simple Hello world LKM
author:         Noel Rodriguez Perez
license:        GPL
srcversion:     2F2B1B95DA1F08AC18B09BC
depends:
vermagic:       4.2.0-42-generic SMP mod_unload modversions

# 3. Consultar las trazas generadas por printk
$localhost:$ dmesg

# 4. Descargar el modulo liberando sus recursos
$localhost:$ sudo rmmod hw
```

Código 2.2: Comandos básicos para la gestión de un *LKM*.

Finalmente, para poder establecer un canal o interfaz de comunicación bidireccional entre el *Espacio de Usuario* y el *Espacio del Kernel*, una de las técnicas más eficientes es la creación de ficheros virtuales a través del pseudo-sistema de ficheros de `/proc`. De esta manera, el módulo instancia un fichero virtual (por ejemplo, `/proc/mydev`) y define manejadores personalizados (*handlers*) que interceptan las llamadas al sistema que actúan sobre él, como `read` o `write`, y así poder declarar específicamente cómo interactuar con el *Espacio de Usuario*. Para implementar esta metodología en el *LKM*, se precisa del uso de funciones específicas de la *API* del *kernel* que aseguran su correcto funcionamiento de forma segura, como `copy_to_user()` y `copy_from_user()` para la transferencia de información entre espacios de memoria distintos [**?**].

En conclusión, el conjunto de todas las características estructurales y funcionales expuestas anteriormente constituye la base tecnológica de un sistema operativo *Linux*.

Como se ha comentado previamente, este ha sido el entorno de trabajo elegido para desarrollar este proyecto, el cual se ha desplegado sobre una máquina física independiente y controlada. Esto permite garantizar un acceso directo a las capas más profundas del sistema operativo, como el *kernel* y sus módulos, mitigando cualquier riesgo sobre máquinas de producción o uso real.

2.1.6. Entorno de desarrollo

Escribe aquí un párrafo explicando brevemente lo que vas a contar en este capítulo. En este capítulo lo ideal es explicar cuáles han sido los objetivos que te has fijado conseguir con tu trabajo, qué requisitos ha de respetar el resultado final, y cómo lo has llevado a cabo; esto es, cuál ha sido tu plan de trabajo.

2.2. Descripción del problema

Cuenta aquí el objetivo u objetivos generales y, a continuación, concrétalos mediante objetivos específicos.

2.3. Requisitos

Describe los requisitos que ha de cumplir tu trabajo.

2.4. Competencias

Enumera las competencias generales adquiridas y empleadas. Es decir, las que consideres que has adquirido durante la realización de tu TFG (las podrás encontrar en la guía docente de la asignatura TFG), así como las que creas que has adquirido en las distintas asignaturas del grado (descritas en sus respectivas guías docentes) y que has empleado para llevarlo a cabo.

2.5. Metodología

Qué paradigma de desarrollo software has seguido para alcanzar tus objetivos.

2.6. Plan de trabajo

Qué agenda has seguido. Si has ido manteniendo reuniones semanales, cumplimentando objetivos parciales, si has ido afinando poco a poco un producto final completo, etc.

Capítulo 3
Plataforma de desarrollo

Quizás algún fragmento de libro inspirador...

Autor, *Título*

Escribe aquí un párrafo explicando brevemente lo que vas a contar en este capítulo. En este capítulo, explica qué has usado a nivel hardware y software para poder desarrollar tu trabajo: librerías, sistemas operativos, plataformas, entornos de desarrollo, etc.

Capítulo 4
Diseño

Quizás algún fragmento de libro inspirador...

Autor, *Título*

Escribe aquí un párrafo explicando brevemente lo que vas a contar en este capítulo. En este capítulo (y quizás alguno más) es donde, por fin, describes detalladamente qué has hecho y qué experimentos has llevado a cabo para validar tus desarrollos.

4.1. Snippets

Puede resultar interesante, para clarificar la descripción, mostrar fragmentos de código (o *snippets*) ilustrativos. En el Código 4.1 vemos un ejemplo escrito en `C++`.

```
void Memory::hypothesizeParallelograms () {
  for(it1 = this->controller->segmentMemory.begin(); it1++) {
    squareFound = false; it2 = it1; it2++;
    while ((it2 != this->controller->segmentMemory.end()) && (!squareFound))
        {
      if (geometry::haveACommonVertex((*it1),(*it2),&square)) {
        dist1 = geometry::distanceBetweenPoints3D ((*it1).start, (*it1).end);
        dist2 = geometry::distanceBetweenPoints3D ((*it2).start, (*it2).end);
      }
    // [...]
```

Código 4.1: Función para buscar elementos 3D en la imagen

En el Código 4.2 vemos un ejemplo escrito en `Python`.

4.2. Verbatim

Para mencionar identificadores usados en el código —como nombres de funciones o variables— en el texto, usa el entorno literal o verbatim

```
def mostrarValores():
    print (w1.get(), w2.get())

master = Tk()
w1 = Scale(master, from_=0, to=42)
w1.pack()
w2 = Scale(master, from_=0, to=200, orient=HORIZONTAL)
w2.pack()
Button(master, text='Show', command=mostrarValores).pack()

mainloop()
```

Código 4.2: Cómo usar un Slider

`hypothesizeParallelograms()`. También se puede usar este entorno para varias líneas, como se ve a continuación:

```
void Memory::hypothesizeParallelograms () {
  // add your code here
}
```

4.3. Ecuaciones

Si necesitas insertar alguna ecuación, puedes hacerlo. Al igual que las figuras, no te olvides de referenciarlas. A continuación se exponen algunas ecuaciones de ejemplo: Ecuación 4.1 y Ecuación 4.2.

$$H = 1 - \frac{\sum_{i=0}^{N} \frac{(\frac{d_{js}+d_{je}}{2})}{N}}{M} \tag{4.1}$$

Ecuación 4.1: Ejemplo de ecuación con fracciones

$$v(entrada) = \begin{cases} 0 & \text{if} \quad \epsilon_t < 0{,}1 \\ K_p \cdot (T_t - T) & \text{if} \quad 0{,}1 \leq \epsilon_t < M_t \\ K_p \cdot M_t & \text{if} \quad M_t < \epsilon_t \end{cases} \tag{4.2}$$

Ecuación 4.2: Ejemplo de ecuación con array y letras y símbolos especiales

4.4. Tablas o cuadros

Si necesitas insertar una tabla, hazlo dígnamente usando las propias tablas de LaTeX, no usando pantallazos e insertándolas como figuras... En el Cuadro 4.1 vemos

un ejemplo.

Parámetros	**Valores**
Tipo de sensor	Sony IMX219PQ[7] CMOS 8-Mpx
Tamaño del sensor	3.674 x 2.760 mm (1/4"format)
Número de pixels	3280 x 2464 (active pixels)
Tamaño de pixel	1.12 x 1.12 um
Lente	f=3.04 mm, f/2.0
Ángulo de visión	62.2 x 48.8 degrees
Lente SLR equivalente	29 mm

Tabla 4.1: Parámetros intrínsecos de la cámara

Capítulo 5
Conclusiones

Quizás algún fragmento de libro inspirador...

Autor, *Título*

Escribe aquí un párrafo explicando brevemente lo que vas a contar en este capítulo, que básicamente será una recapitulación de los problemas que has abordado, las soluciones que has prouesto, así como los experimentos llevados a cabo para validarlos. Y con esto, cierras la memoria.

5.1. Conclusiones

Enumera los objetivos y cómo los has cumplido.

Enumera también los requisitos implícitos en la consecución de esos objetivos, y cómo se han satisfecho.

No olvides dedicar un par de párrafos para hacer un balance global de qué has conseguido, y por qué es un avance respecto a lo que tenías inicialmente. Haz mención expresa de alguna limitación o peculiaridad de tu sistema y por qué es así. Y también, qué has aprendido desarrollando este trabajo.

Por último, añade otro par de párrafos de líneas futuras; esto es, cómo se puede continuar tu trabajo para abarcar una solución más amplia, o qué otras ramas de la investigación podrían seguirse partiendo de este trabajo, o cómo se podría mejorar para conseguir una aplicación real de este desarrollo (si es que no se ha llegado a conseguir).

5.2. Corrector ortográfico

Una vez tengas todo, no olvides pasar el corrector ortográfico de LaTeXa todos tus ficheros *.tex*. En `Windows`, el propio editor `TeXworks` incluye el corrector. En `Linux`, usa `aspell` ejecutando el siguiente comando en tu terminal:

```
aspell --lang=es --mode=tex check capitulo1.tex
```